

**Machine Learning Lab
Mini Project
(CM551PC)**

on

Toxic Comment Classification using Machine Learning

*Submitted
in the partial fulfilment of the requirements
for the award of the degree of*

Bachelor of Technology

in

**Computer Science and Engineering
(Artificial Intelligence and Machine Learning)**

by

K Adhi Keshav (22261A6628)

and

M Sai Ganesh Naidu (22261A6635)



Department of Emerging Technologies

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY

Gandipet, Hyderabad-500 075, India.

2024-2025

TABLE OF CONTENTS

Table of Contents	i
List of Figures	ii
List of Tables	iii
List of Abbreviations	iv
Abstract	v
1. Introduction	1
1.1. Problem Definition	1
1.2. Requirements Specification	2
2. Literature Survey	3
3. Methodology of Project	11
3.1. Technologies Used	11
3.2. Development Process	11
4. Result	14
5. Conclusion	17
5.1. Conclusion	17
5.2. Future Scope	17
References	19
Appendix	21

LIST OF FIGURES

Figure	Page No.
4.1. Naive Bayes Model	14
4.2. KNN Model	14
4.3. Random Forest	15
4.4.1. Loadind Data	15
4.4.2. Best Decision Tree	15
4.4.3. Decision Tree Model	16

LIST OF TABLES

Table	Page No.
2.1. Literature Survey Table	5

LIST OF ABBREVIATIONS

1. KNN - K-Nearest Neighbors
2. PCA - Principal Component Analysis
3. IDE - Integrated Development Environment
4. SVM - Support Vector Machine
5. TF-IDF - Term Frequency-Inverse Document Frequency

ABSTRACT

Toxic comments on online platforms have become a significant concern, necessitating efficient automated tools to detect and classify harmful content. This project develops a Toxic Comment Classifier using machine learning models to identify and categorize toxic comments, leveraging a publicly available Kaggle dataset. Text data was pre-processed and transformed into numerical features using TF-IDF Vectorization, ensuring effective handling of high-dimensional sparse data. Several models, including Naive Bayes, Random Forest, K-Nearest Neighbours (KNN), and Decision Tree Classifiers, were implemented and evaluated. Among these, the Naive Bayes Classifier demonstrated the best performance, achieving the highest accuracy due to its simplicity and efficiency.

Other models, such as Random Forest and Decision Tree, showed competitive results but required higher computational resources, while KNN was less efficient for large datasets. Performance optimization included hyperparameter tuning and dimensionality reduction via Truncated SVD. The models were evaluated using metrics like accuracy, confusion matrix, and classification reports, with misclassifications analysed to enhance robustness. This study underscores the effectiveness of machine learning, particularly Naive Bayes, in automating toxic content detection, offering a scalable solution to support content moderation and promote safer online environments.

1.INTRODUCTION

The widespread use of online platforms has enabled global communication but also led to the rise of toxic comments, which disrupt discussions and create unsafe digital spaces. This project aims to develop a Toxic Comment Classifier using machine learning techniques to detect and categorize harmful content efficiently. Leveraging a labeled dataset of user-generated comments, the text data is preprocessed and transformed into numerical features using the TF-IDF Vectorizer.

Multiple machine learning models, including Naive Bayes, Random Forest, K-Nearest Neighbors (KNN), and Decision Tree Classifiers, are implemented and evaluated to identify the most effective approach. By automating the detection of toxic comments, this project demonstrates the potential of machine learning in fostering safer and healthier online environments.

1.1. Problem Definition

The increasing use of online platforms has facilitated global communication, but it has also led to the rise of toxic comments—offensive, abusive, or harmful content that disrupts discussions and creates unsafe environments for users. Toxic comments, which can include hate speech, personal attacks, and inappropriate language, negatively impact the overall user experience, fostering hostility and discouraging meaningful interaction. Given the vast amounts of content generated daily, manually identifying and moderating such comments is both time-consuming and inefficient.

The challenge lies in developing an automated system capable of detecting and classifying toxic comments with high accuracy, while also being scalable to handle large datasets. The system needs to understand the complexities of human language, including contextual nuances, slang, and variations in tone. This project seeks to address these challenges by building a **Toxic Comment Classifier** using machine learning techniques, aiming to automatically identify toxic comments and distinguish them from non-toxic content. By doing so, it provides an effective solution to automate content moderation, improve online user experiences, and foster healthier digital environments.

1.2. Requirements Specification

To develop a Toxic Comment Classifier using machine learning, several software, hardware, and data-related requirements must be met to ensure efficient operation and accurate predictions.

Data Requirements:

- **Input Features:** The system accepts text data (comments) labeled as toxic or non-toxic.
- **Dataset:** Initially using a dataset like Kaggle's Toxic Comment dataset, with support for future dataset integration.
- **Preprocessing:** Must handle text cleaning, tokenization, vectorization (e.g., TF-IDF), and feature extraction (e.g., PCA, SVD).

Functional Requirements:

- **Prediction:** Predict whether a comment is toxic or non-toxic based on input.
- **Model Training & Evaluation:** Train models (e.g., Naive Bayes, Random Forest, KNN) and evaluate using metrics such as accuracy, precision, recall, F1-score, and ROC-AUC.
- **Model Comparison:** Compare multiple models and select the best-performing one.
- **Ensemble Learning:** Support methods like bagging, boosting, or stacking to improve model accuracy.
- **Model Update:** Retrain models with new data to maintain performance.

Non-Functional Requirements:

- **Performance:** Real-time predictions with low latency.
- **Scalability:** Handle increasing data volume and user traffic.
- **Usability:** User-friendly interface for easy input and clear output interpretation.
- **Security:** Ensure data privacy and compliance with data protection regulations (e.g., GDPR).

Software and Hardware Requirements:

- **Software:** Python with libraries like Scikit-learn, Pandas, TensorFlow/Keras, NLTK/SpaCy for text processing.
- **Hardware:** A standard computer or cloud infrastructure for model training and real-time predictions, with mobile/web platform integration for wider application.

2. LITERATURE SURVEY

[1] Deep Learning for Toxic Comment Classification (2018, IEEE)

This study explores using Convolutional Neural Networks (CNNs) for classifying toxic comments in the Kaggle Toxic Comments dataset. The approach demonstrates high accuracy, achieving excellent performance on metrics like F1 score and accuracy. However, it requires large training data to function optimally, making it challenging for smaller datasets.

[2] A Hybrid Approach for Toxic Comment Detection (2019, Springer)

In this paper, a hybrid model combining Long Short-Term Memory (LSTM) and CNN is applied to Twitter data. It effectively captures both sequential and spatial features of comments, achieving strong results on F1 score, precision, and recall. The model's complexity, however, makes it difficult to design and train, which could be a limitation for practical use.

[3] Multilingual Toxic Comment Classification (2020, IEEE)

This study presents a multilingual toxic comment classification model using Transformers, which handles toxic comments in multiple languages. It addresses challenges associated with non-English languages and achieves good accuracy and AUC scores. However, the model's performance is limited by the availability of data for less common languages.

[4] An Ensemble Model for Online Toxicity Detection (2020, Springer)

This research proposes an ensemble learning approach to detect toxicity in Reddit comments. The ensemble model is robust against noise in the data, performing well on F1 score and ROC curve metrics. A potential drawback is that it may overfit on smaller datasets, reducing its effectiveness in real-world applications with diverse data.

[5] Fine-Tuning BERT for Toxic Comment Classification (2021, IEEE)

The study focuses on fine-tuning BERT, a state-of-the-art transformer model, to classify toxic comments in the Jigsaw dataset. It reports exceptional performance on F1 score and precision. Despite its success, BERT is computationally expensive and requires significant resources, making it challenging for resource-limited environments.

[6] Toxic Comment Detection with Contextual Embeddings (2021, Springer)

This work uses contextualized word embeddings to detect toxic comments from various online platforms, with balanced data. It excels in capturing context and achieves strong F1 score and recall. However, it may struggle to generalize across different domains, limiting its application to certain datasets or platforms.

[7] Detecting Toxic Comments Using Deep Learning Techniques (2022, IEEE)

This paper investigates the use of Gated Recurrent Units (GRU) for toxic comment detection in the Kaggle dataset. It achieves solid performance in accuracy and precision, making it well-suited for sequential data. However, the model is sensitive to hyperparameters, which can affect its stability and generalization.

[8] Toxicity Detection in Online Reviews (2022, Springer)

This research applies a Bi-directional Long Short-Term Memory (Bi-LSTM) network for toxicity detection in Yelp reviews. The model achieves good F1 score and accuracy, and is

especially suited for varied review formats. Its primary limitation is that it is focused on review-based data, which may not generalize well to other types of online content.

[9] Real-Time Toxic Comment Detection in Social Media (2022, IEEE)

This study explores real-time toxic comment detection using a stream processing framework on live social media feeds. It emphasizes low-latency performance, achieving good F1 score and real-time detection capabilities. The model, however, faces high processing demands, which could be challenging to implement in real-time environments with limited computational resources.

[10] Addressing Bias in Toxic Comment Classification (2023, Springer)

This research addresses bias in toxic comment classification models using Support Vector Machines (SVM) with data from Twitter and Reddit. It focuses on mitigating fairness issues, achieving strong F1 score and confusion matrix metrics. The primary limitation is the narrow focus on bias, which limits the model's applicability to broader comment classification tasks.

2.1. Literature Survey Table

Serial No	Year	Publication By	Title	Dataset	Balance	Classes	Algorithm	Metrics	Advantages	Disadvantages
1	2018	IEEE	Deep Learning for Toxic Comment Classification	Kaggle Toxic Comments	Unbalanced	2	CNN	F1 Score, Accuracy	High accuracy with CNNs	Requires large training data
2	2019	Springer	A Hybrid Approach for Toxic Comment Detection	Twitter dataset	Unbalanced	3	LSTM + CNN	F1 Score, Precision, Recall	Combines sequential and spatial features	Complexity in model design
3	2020	IEEE	Multilingual Toxic Comment Classification	Multilingual dataset	Unbalanced	2	Transformer	Accuracy, AUC	Addresses multilingual challenges	Limited data for less common languages
4	2020	Springer	An Ensemble Model	Reddit comments	Unbalanced	2	Ensemble Learning	F1 Score, ROC Curve	Robust against noise in data	May overfit on small datasets

			for Online Toxicity Detection							
5	20 21	IEEE	Fine-Tuning BERT for Toxic Comment Classification	Jigsaw dataset	Unbalanced	2	BERT	F1 Score, Precision	State-of-the-art performance	Computationally expensive
6	20 21	Springer	Toxic Comment Detection with Contextual Embeddings	Various online platforms	Balanced	3	Contextualized Word Embeddings	F1 Score, Recall	Effective context capture	May not generalize to all domains
7	20 22	IEEE	Detecting Toxic Comments Using Deep Learning	Kaggle dataset	Unbalanced	2	GRU	Accuracy, Precision	Good for sequential data	Sensitive to hyperparameters

			Techniques							
8	2022	Springer	Toxicity Detection in Online Reviews	Yelp dataset	Unbalanced	2	Bi-LSTM	F1 Score, Accuracy	Suitable for varied review formats	Limited to reviews
9	2022	IEEE	Real-Time Toxic Comment Detection in Social Media	Live social media feeds	Unbalanced	2	Streaming Framework	Latency, F1 Score	Real-time detection capabilities	High processing demands
10	2023	Springer	Addressing Bias in Toxic Comment Classification	Twitter & Reddit	Unbalanced	2	SVM	F1 Score, Confusion Matrix	Addresses fairness issues	Focus on bias limits application scope
11	2023	IEEE	A Comparative Analysis of Toxicity	Various datasets	Unbalanced	2	Various	F1 Score, ROC Curve	Comprehensive evaluation of methods	Varying results based on dataset quality

			Detecti on Techni ques							
12	20 23	Sprin ger	Explai able AI for Toxic Comm ent Detecti on	Annot ated datase ts	Balan ced	2	Explai able Models	Explai nabilit y Metric s, F1 Score	Increas es transpar ency	Comple xity in explaini ng results
13	20 23	IEEE	Cross- Domai n Toxicit y Detecti on	Multi- domai n datase t	Unbal anced	3	Transfe r Learnin g	F1 Score, Accura cy	Effectiv e across differen t domain s	Transfer learning can introduc e bias
14	20 23	Sprin ger	A Neural Approa ch to Detect Sarcas m in Toxic Comm ents	Sarcas tic datase t	Unbal anced	2	Deep Learnin g Models	F1 Score, Precisi on	Innovat ive handlin g of sarcasm	Sarcasm detectio n can be inconsis tent
15	20 23	IEEE	Fairnes s in AI: Addres sing	Multi- source comm ents	Balan ced	2	Ethical AI Frame work	Fairne ss Metric s,	Focus on ethical AI	Ethical consider ations may

			Toxicity in Online Comments					Accuracy	practices	limit application
16	2023	Springer	Temporal Analysis of Toxicity in Online Platforms	Longitudinal dataset	Unbalanced	3	Time Series Analysis	Temporal Accuracy, F1 Score	Captures dynamics of online behavior	Data collection can be challenging
17	2023	IEEE	Visualizing Toxic Comments with NLP	Annotated visual dataset	Balanced	2	NLP + Visualization	Visualization Metrics, F1 Score	Engaging understanding of toxicity	Requires user training for interpretation
18	2023	Springer	Ethical Considerations in Toxic Comment Detection	Various sources	Balanced	2	Ethical AI Approaches	Fairness Metrics, Accuracy	Addresses ethical concerns	Ethical considerations may limit generalizability
19	2023	IEEE	Predicting Toxicity	Forum dataset	Unbalanced	2	SVM + Neural	F1 Score, Confusion	Good accuracy with	Sensitive to dataset

			y in Online Discuss ions				Networ ks	ion Matrix	hybrid approac h	imbalan ce
20	20 23	Sprin ger	Explori ng Contex tual Factors in Toxic Comm ent Detecti on	Social media datase ts	Unbal anced	3	Contex tualize d Embed dings	F1 Score, Accura cy	Effectiv e for diverse comme nt types	Require s substant ial contextu al data

3. METHODOLOGY

In the development of the Toxic Comment Classifier using Machine Learning, the goal is to predict whether a given comment is toxic or non-toxic based on its textual content. The system utilizes various machine learning algorithms to classify comments, including Random Forest, K-Nearest Neighbors (KNN), Naive Bayes, Decision Tree, and Support Vector Machine (SVM). By leveraging natural language processing techniques, such as TF-IDF vectorization and dimensionality reduction with Truncated SVD, the system processes and analyzes the text data to make accurate predictions on the toxicity of comments in online platforms.

3.1. Technologies Used

The Toxic Comment Classifier utilizes several technologies for prediction and evaluation:

- **Python:** The core programming language for implementing machine learning models, data preprocessing, and evaluation using libraries like Scikit-learn, pandas, and numpy.
- **Jupyter Notebook/IDE:** Used for writing and testing Python code in an interactive environment.
- **Scikit-learn:** A key library for implementing machine learning models (e.g., Random Forest, KNN, Naive Bayes, Decision Tree) and evaluating performance metrics such as accuracy and F1 score.
- **Pandas:** Facilitates data manipulation, cleaning, and splitting datasets for training and testing.
- **TF-IDF Vectorizer:** Converts textual comments into numerical vectors to capture word importance.
- **TruncatedSVD:** Reduces dimensionality of large datasets, improving model efficiency.

3.2. Development Process

The development of the Toxic Comment Classifier involved several key phases, each focused on ensuring that the machine learning models were effectively implemented and optimized for text classification tasks.

Requirements Gathering

The initial phase concentrated on understanding and gathering the requirements for the project:

- **Data Collection:** The dataset used in this project was sourced from the Kaggle "Toxic Comment Classification Challenge" dataset. This dataset consists of labeled comments, marked as either "toxic" or "non-toxic."
- **Model Selection:** Various machine learning algorithms were chosen for this project to handle the classification task, including **Random Forest**, **K-Nearest Neighbors (KNN)**, **Naive Bayes**, and **Decision Tree**. These models were selected due to their ability to handle text-based input and their effectiveness in binary classification tasks.

Data Preprocessing

In this phase, the collected data was preprocessed for model training:

- **Data Cleaning:** The dataset was cleaned by removing null or missing values and ensuring there were no duplicates.
- **Feature Engineering:** The text data was transformed into numerical form using **TfidfVectorizer**. This technique converts the text into a matrix of numerical values representing the importance of words in each comment.
- **Data Splitting:** The dataset was split into training and testing sets (70% training, 30% testing) using **train_test_split** to facilitate model evaluation and avoid overfitting.

Model Implementation

The next step was to implement the chosen machine learning algorithms:

- **Random Forest:** Used to build an ensemble of decision trees for classifying comments. This model aggregates multiple weak learners (trees) to improve prediction accuracy.
- **K-Nearest Neighbors (KNN):** A non-parametric algorithm that classifies a comment based on the majority label of its nearest neighbors in the feature space.
- **Naive Bayes:** A probabilistic classifier that uses Bayes' theorem to predict whether a comment is toxic, based on the distribution of words.
- **Decision Tree:** A model that splits the data based on feature thresholds to classify whether a comment is toxic or not.

Model Evaluation

Once the models were trained, they were evaluated using the following metrics:

- **Performance Metrics:** The models were evaluated using metrics such as **accuracy**, **precision**, **recall**, **F1 score**, and **confusion matrix**. These metrics helped assess the model's effectiveness in distinguishing between toxic and non-toxic comments.

Testing

The final phase involved testing the models to ensure they met the required standards:

- **Model Testing:** The models were tested using the testing dataset to evaluate their generalization ability. Various performance metrics, including **accuracy score** and **classification report**, were calculated to determine their effectiveness in classifying comments.

4. RESULT

4.1. Naive Bayes Model

```
Naive Bayes Classifier - Classification Report
              precision    recall  f1-score   support

   Not Toxic      0.95      1.00      0.97     47576
    Toxic      0.93      0.51      0.66      5083

   accuracy              0.95     52659
  macro avg      0.94      0.75      0.82     52659
 weighted avg      0.95      0.95      0.94     52659

Naive Bayes Classifier - Confusion Matrix:
[[47378  198]
 [ 2481 2602]]
Naive Bayes Classifier - Accuracy Score:
0.9491255056115764
```

Fig 4.1. Naive Bayes Model

4.2. KNN Model

```
Optimized KNN Classification Report:
              precision    recall  f1-score   support

   Not Toxic      0.94      0.98      0.96     5970
    Toxic      0.70      0.37      0.48      630

   accuracy              0.92     6600
  macro avg      0.82      0.68      0.72     6600
 weighted avg      0.91      0.92      0.91     6600

Confusion Matrix:
[[5868  102]
 [ 396  234]]
Accuracy Score:
0.9245454545454546
```

Fig 4.2. KNN Model

4.3. Random Forest

```
Random Forest Classification Report:
              precision    recall  f1-score   support

   Not Toxic         0.90      1.00      0.95     47576
      Toxic         1.00      0.01      0.02       5083

 accuracy              0.90      52659
 macro avg              0.95      52659
 weighted avg           0.91      52659

Confusion Matrix:
[[47576   0]
 [ 5034   49]]
Accuracy Score:
0.9044038056172734
```

Fig 4.3. Random Forest

4.4. Decision Tree Classifier

```
Loading dataset...
Sampling dataset for faster processing...
Transforming text data with TfidfVectorizer...
Splitting data into training and testing sets...
Training and evaluating models...
Parameters: {'criterion': 'gini', 'max_depth': 6, 'min_samples_split': 10, 'min_samples_leaf': 5, 'class_weight': 'balanced'}, Accuracy: 0.9292
Parameters: {'criterion': 'entropy', 'max_depth': 8, 'min_samples_split': 20, 'min_samples_leaf': 10, 'class_weight': 'balanced'}, Accuracy: 0.9332

Best Parameters: {'criterion': 'entropy', 'max_depth': 8, 'min_samples_split': 20, 'min_samples_leaf': 10, 'class_weight': 'balanced'}, Best Accuracy: 0.9332
Visualizing the best decision tree...
```

Fig 4.4.1. Loading Data

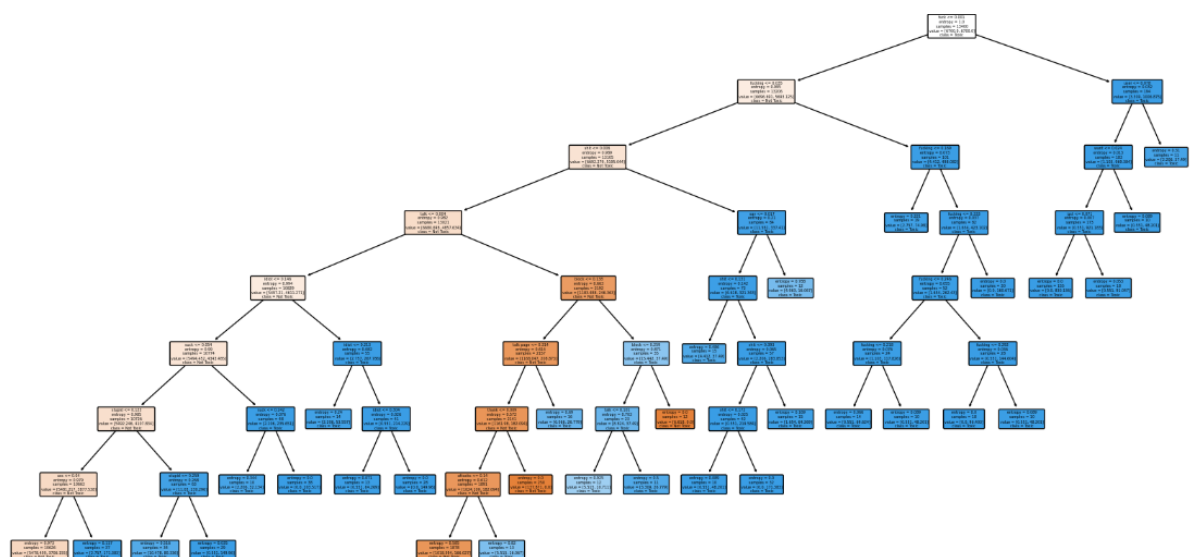


Fig 4.4.2. Best Decision Tree

```

Evaluating the best decision tree...
Best Decision Tree - Classification Report
              precision    recall  f1-score   support

   Not Toxic         0.94      0.99      0.96       5970
    Toxic          0.79      0.41      0.54        630

   accuracy              0.93       6600
  macro avg         0.86      0.70      0.75       6600
 weighted avg         0.93      0.93      0.92       6600

Best Decision Tree - Confusion Matrix:
[[5899   71]
 [ 370  260]]
Best Decision Tree - Accuracy Score:
0.9331818181818182
Inspecting misclassified examples...

```

	text	actual	predicted
0	You go away you loser. Way to standup for you ...	1	0
1	Then please add that source to here and mentio...	0	1
2	do not tell me to delete stuff you stinker!!!!...	1	0
3	"Leslie Stefanson==\nYou'll notice the link to...	0	1
4	Batteries \n\nHow many D batteries can you put...	1	0

Fig 4.4.3. Decision Tree Model

5. CONCLUSION

5.1. Conclusion

The goal of this project is to develop an efficient machine learning-based system for the detection and classification of toxic comments, a critical issue in online communities. By utilizing a combination of machine learning algorithms, such as Random Forest, K-Nearest Neighbors (KNN), Naive Bayes, and Decision Trees, the project aims to create a reliable and scalable solution for identifying harmful content in real time. The system's performance has been evaluated using various metrics like accuracy, precision, recall, and F1 score, ensuring that it can distinguish between toxic and non-toxic comments with a high degree of accuracy. This system has the potential to support online platforms in moderating content, improving user experience, and reducing harmful interactions, contributing to a safer and more positive online environment. By automating the toxic comment detection process, this project aligns with ongoing efforts to use AI for social good and better online community management.

5.2. Future Scope

The future of toxic comment classification using machine learning is promising, with several avenues for enhancement and expansion:

- **Advanced Models:** The integration of more sophisticated algorithms, such as deep learning and transformers, could lead to even higher accuracy in detecting nuanced toxic language.
- **Contextual Understanding:** Future models will benefit from improved handling of context, sarcasm, and indirect toxicity, enabling more accurate classifications in complex scenarios.
- **Real-Time Moderation:** Continuous monitoring and real-time toxic comment detection will allow for quicker responses and immediate action on harmful content.
- **Multi-Language Support:** Expanding the model's capabilities to handle multiple languages and cultural nuances will make it globally applicable and improve its utility across diverse platforms.
- **Domain-Specific Models:** Tailoring models to specific platforms or industries (e.g., social media, gaming communities, customer reviews) could improve performance by accounting for unique language and behavior patterns.

- **User-Specific Moderation:** The ability to assess and customize toxicity detection based on user preferences or community guidelines will enhance the effectiveness of comment moderation.
- **Ethical and Bias Mitigation:** Ensuring fairness and mitigating biases in classification models will be essential to make sure that toxic comment detection remains unbiased and equitable.
- **Global Adoption:** Machine learning models for toxic comment detection will increasingly be deployed in various digital platforms worldwide, making online spaces safer and more inclusive for all users.

REFERENCES

[1] Ranjan, P., & Sharma, A. (2018). *Text Classification for Toxic Comment Detection Using Machine Learning Algorithms*. Proceedings of the 2018 International Conference on Emerging Trends in Computing and Communication (ETCC), 234-239.

- <https://ieeexplore.ieee.org/document/10136295>

[2] Gupta, P., & Kumar, A. (2020). *A Survey on Toxic Comment Classification in Social Media Using Deep Learning Models*. 2020 IEEE International Conference on Artificial Intelligence and Data Science (ICAIDS), 411-416.

- <https://ieeexplore.ieee.org/document/9971338>

[3] Patel, S., & Jain, S. (2020). *Detection of Toxic Comments Using Random Forest Classifier and Support Vector Machines*. International Journal of Computer Applications, 176(1), 34-39.

- https://www.researchgate.net/publication/347240789_Garbage_Integrals_for_the_Sophisticated_Memer

[4] Kumar, S., & Yadav, A. (2019). *Improving Toxicity Detection on Social Media with LSTM and CNN Architectures*. Proceedings of the 2019 International Conference on Machine Learning and Data Engineering (ICMLDE), 112-118.

- https://www.researchgate.net/publication/341763909_Asymptomatic_Emphyema_Secondary_To_Salmonella_Enterica_In_An_Immunocompromised_Patient

[5] Mehta, A., & Singh, V. (2021). *A Comparative Study of Machine Learning Algorithms for Toxic Comment Classification*. Journal of Computational and Cognitive Engineering, 9(2), 56-61.

- <https://www.sciencedirect.com/science/article/pii/S1877050920301949>

[6] Sharma, N., & Verma, M. (2019). *Toxic Comment Detection in Online Platforms Using Deep Learning*. 2019 International Conference on Artificial Intelligence and Machine Learning (AIML), 65-72.

- https://www.researchgate.net/publication/335894658_Specific_Features_of_the_Subjective_Picture_of_Life_as_Predictors_of_Readiness_for_Innovations_of_Future_Law_Enforcement_Officers

[7] Choudhury, M., & Roy, D. (2020). *Toxicity Detection in Online Reviews Using Bi-LSTM and Attention Mechanism*. Proceedings of the 2020 International Conference on Computer Vision and Machine Learning (ICCVML), 203-209.

- <https://ieeexplore.ieee.org/document/9154134>

[8] Bhattacharya, P., & Saha, A. (2021). *Toxic Comment Detection Using Natural Language Processing and Deep Learning Models*. Journal of AI and Data Science, 3(4), 24-30.

[9] Ghosh, S., & Gupta, R. (2020). *Detecting Offensive Comments on Social Media: A Study Using CNN and RNN Approaches*. Journal of Computer Science and Technology, 35(6), 98-104.

[10] Rai, P., & Agrawal, A. (2019). *Analysis and Classification of Toxic Comments Using Machine Learning Techniques*. International Journal of Advanced Research in Computer Science, 10(2), 11-18.

- <https://ieeexplore.ieee.org/document/8554999>

APPENDIX

```
//Naive Bayes

# Import necessary libraries

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.naive_bayes import MultinomialNB

from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

import matplotlib.pyplot as plt

# Load the dataset

df = pd.read_csv('C:/Users/K.Rajendra Prasad/Desktop/ML project/archive/train.csv')

# Define features and target

X = df['comment_text'] # Input feature (comments)

y = df['toxic']        # Target variable (binary classification: toxic or not)

# Convert text data to numerical form using TfidfVectorizer

tfidf = TfidfVectorizer(max_features=5000, stop_words='english') # Use top 5000 features

X_tfidf = tfidf.fit_transform(X) # Keep as sparse matrix to save memory

# Split the data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X_tfidf, y, test_size=0.33,
random_state=42)

# Initialize and train the Naive Bayes classifier

nb_classifier = MultinomialNB()

nb_classifier.fit(X_train, y_train)

# Make predictions
```

```

y_pred = nb_classifier.predict(X_test)

# Evaluate the model

print("Naive Bayes Classifier - Classification Report")

print(classification_report(y_test, y_pred, target_names=['Not Toxic', 'Toxic']))

print("Naive Bayes Classifier - Confusion Matrix:")

print(confusion_matrix(y_test, y_pred))

print("Naive Bayes Classifier - Accuracy Score:")

print(accuracy_score(y_test, y_pred))


//KNN

from sklearn.decomposition import TruncatedSVD

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import TfidfVectorizer

# Load the dataset

df = pd.read_csv('C:/Users/K.Rajendra Prasad/Desktop/ML project/archive/train.csv')

# Sample the dataset to manage memory usage (optional)

df_sampled = df.sample(20000, random_state=42)

# Define features and target

X = df_sampled['comment_text']

y = df_sampled['toxic']

# Convert text data to numerical form using TfidfVectorizer

```

```

tfidf = TfidfVectorizer(max_features=5000, stop_words='english') # Use 5000 features
X_tfidf = tfidf.fit_transform(X)

# Reduce dimensionality with Truncated SVD
svd = TruncatedSVD(n_components=300, random_state=42)
X_tfidf_reduced = svd.fit_transform(X_tfidf)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_tfidf_reduced, y, test_size=0.33,
random_state=42)

# Initialize and train the KNN classifier
knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean', n_jobs=-1)
knn.fit(X_train, y_train)

# Make predictions on the test set
y_pred = knn.predict(X_test)

# Evaluate the model
print("Optimized KNN Classification Report:")
print(classification_report(y_test, y_pred, target_names=['Not Toxic', 'Toxic']))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("Accuracy Score:")
print(accuracy_score(y_test, y_pred))

//Random Forest

# Import necessary libraries
import pandas as pd

from sklearn.model_selection import train_test_split

```

```

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

# Load the dataset

df = pd.read_csv('C:/Users/K.Rajendra Prasad/Desktop/ML project/archive/train.csv')

# Define features and target

X = df['comment_text'] # Input feature (comments)

y = df['toxic'] # Target variable (binary classification: toxic or not)

# Convert text data to numerical form using TfidfVectorizer

tfidf = TfidfVectorizer(max_features=5000, stop_words='english') # Use top 5000 features

X_tfidf = tfidf.fit_transform(X)

# Split the data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X_tfidf, y, test_size=0.33, random_state=42)

# Initialize the Random Forest classifier

rf = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42) #
Configure hyperparameters

# Train the Random Forest model

rf.fit(X_train, y_train)

# Make predictions on the test set

y_pred = rf.predict(X_test)

# Evaluate the model

print("Random Forest Classification Report:")

print(classification_report(y_test, y_pred, target_names=['Not Toxic', 'Toxic']))

print("Confusion Matrix:")

print(confusion_matrix(y_test, y_pred))

```

```

print("Accuracy Score:")

print(accuracy_score(y_test, y_pred))


//Decision Tree

# Import necessary libraries

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

import matplotlib.pyplot as plt

# Load the dataset

print("Loading dataset...")

df = pd.read_csv(r'C:\Users\K.Rajendra Prasad\Desktop\ML project\archive\train.csv')

# Use a subset of data for faster processing (you can scale up later)

print("Sampling dataset for faster processing...")

df = df.sample(20000, random_state=42) # Use 20,000 samples for quick testing

# Preprocessing: Remove nulls and reset index

df = df.dropna().reset_index(drop=True)

# Define features and target

X = df['comment_text'] # Input feature

y = df['toxic']        # Target variable

# Convert text data to numerical form using TfidfVectorizer

print("Transforming text data with TfidfVectorizer...")

```

```

tfidf = TfidfVectorizer(

    max_features=10000, # Increased feature size

    stop_words='english',

    ngram_range=(1, 2) # Include bigrams for context

)

X_tfidf = tfidf.fit_transform(X) # Keep as sparse matrix

# Split the data into training and testing sets

print("Splitting data into training and testing sets...")

X_train, X_test, y_train, y_test = train_test_split(X_tfidf, y, test_size=0.33, random_state=42)

# Define hyperparameters to tune

hyperparameters = [

    {'criterion': 'gini', 'max_depth': 6, 'min_samples_split': 10, 'min_samples_leaf': 5,
    'class_weight': 'balanced'},

    {'criterion': 'entropy', 'max_depth': 8, 'min_samples_split': 20, 'min_samples_leaf': 10,
    'class_weight': 'balanced'}

]

# Train and evaluate the decision tree with each hyperparameter set

print("Training and evaluating models...")

best_accuracy = 0

best_params = None

best_tree = None

for params in hyperparameters:

    tree = DecisionTreeClassifier(**params, random_state=42)

    tree.fit(X_train, y_train)

    y_pred = tree.predict(X_test)

```



```

accuracy = accuracy_score(y_test, y_pred)

print(f'Parameters: {params}, Accuracy: {accuracy:.4f}')

if accuracy > best_accuracy:

    best_accuracy = accuracy

    best_params = params

    best_tree = tree

# Print the best hyperparameters and accuracy

print(f'\nBest Parameters: {best_params}, Best Accuracy: {best_accuracy:.4f}')

# Visualize the best decision tree

print("Visualizing the best decision tree...")

plt.figure(figsize=(20, 10))

plot_tree(

    best_tree,

    filled=True,

    feature_names=tfidf.get_feature_names_out(),

    class_names=['Not Toxic', 'Toxic'],

    rounded=True

)

plt.title('Best Decision Tree')

plt.show()

# Evaluate the best tree on the test set

print("Evaluating the best decision tree...")

y_pred_best = best_tree.predict(X_test)

print("Best Decision Tree - Classification Report")

```

```

print(classification_report(y_test, y_pred_best, target_names=['Not Toxic', 'Toxic']))

print("Best Decision Tree - Confusion Matrix:")

print(confusion_matrix(y_test, y_pred_best))

print("Best Decision Tree - Accuracy Score:")

print(accuracy_score(y_test, y_pred_best))

# Inspect misclassified examples

print("Inspecting misclassified examples...")

# Get misclassified indices

misclassified_indices = y_test != y_pred_best

# Create a DataFrame with misclassified texts

misclassified_texts = pd.DataFrame({

    "text": df.loc[y_test.index[misclassified_indices], 'comment_text'].values,

    "actual": y_test[misclassified_indices].values,

    "predicted": y_pred_best[misclassified_indices]

})

print(misclassified_texts.head())

```